

MATH 212

HOMEWORK DUE WEEK 9 FRIDAY SOLUTION

Winter 2026

Prof. Chowdhury

Note: Each question is worth 6 points.

Some of the questions requires you to write scripts. Please create a single notebook file that includes the code for all the question.

I might add one more problem at the end, but if I do, that will act as a replacement for the last lab.

■ Question 9001. □

In this exercise, we are going to examine a very common scientific situation: we have data from an experiment and a (challenging to solve analytically) differential equation modeling the data that has some unknown parameters. We want to find the value of the parameter that gives us the best fit between our numerical solution and the experimentally observed data.

As an example, let $T(t)$ denote the temperature of an object in a vacuum chamber at time t . By the Stefan-Boltzmann Law of Radiative Cooling, the rate of change of the temperature is proportional to the difference between the fourth powers of the object's temperature and the constant ambient temperature T_{env} . This is modeled by the initial value problem:

$$\frac{dT}{dt} = -k[T(t)^4 - T_{env}^4], \quad T(t_0) = T_0$$

where $k > 0$ is an unknown constant specific to the object's radiative properties.

Assume an ambient temperature of $T_{env} = 3.0$. You are provided with a comma-separated values (CSV) file, `radiative_cooling_data.csv` in Canvas. This file contains a dataset $\mathcal{D} = \{(t_i, T_i)\}_{i=0}^N$ of experimental observations of the object's temperature over time.

- (a) Import the experimental data from `radiative_cooling_data.csv` into your computational environment.*
- (b) Use the RK4 routine you created in the last exercise to approximate the solution to the IVP (that uses only the experimental observation at t_0).*

Your implementation must accept the parameter k as its sole input and output a discrete sequence of approximated temperatures $y_i(k) \approx T(t_i)$ corresponding to the time steps t_i in the dataset.

- (c) Experiment with different k values and plot the approximations against the experimental dataset $\mathcal{D}$ until they somewhat resemble each other. You probably want to start around 10^{-5} .*

Based on these results, establish an interval $[k_{\min}, k_{\max}]$ that most likely contains the true parameter k .

- (d) Formulate the objective function $S : \mathbb{R}^+ \rightarrow \mathbb{R}$ defined by the sum of squared residuals between the numerical approximation and the experimental data:*

$$S(k) = \sum_{i=0}^N (y_i(k) - T_i)^2$$

Implement a script to evaluate $S(k)$ for a list of k values in $[k_{\min}, k_{\max}]$ with appropriately chosen Δk . Evaluate and plot $S(k)$ over your chosen interval to verify convexity in this region.

- (e) Utilize an established numerical optimization routine from a standard library to minimize $S(k)$.

Note: In Python, you can use `scipy.optimize.minimize` and in Mathematica, you can use `FindMinimum`.

- (f) Report the optimal parameter k^* . Superimpose your optimized numerical solution $y_i(k^*)$ onto the scatter plot of the experimental dataset to visually verify the fit.

At this point you can now use the best numerical solution to answer questions about the scientific setup (e.g. extrapolation).

- (g) Approximately how long will it take for the temperature to be reduced to 5?

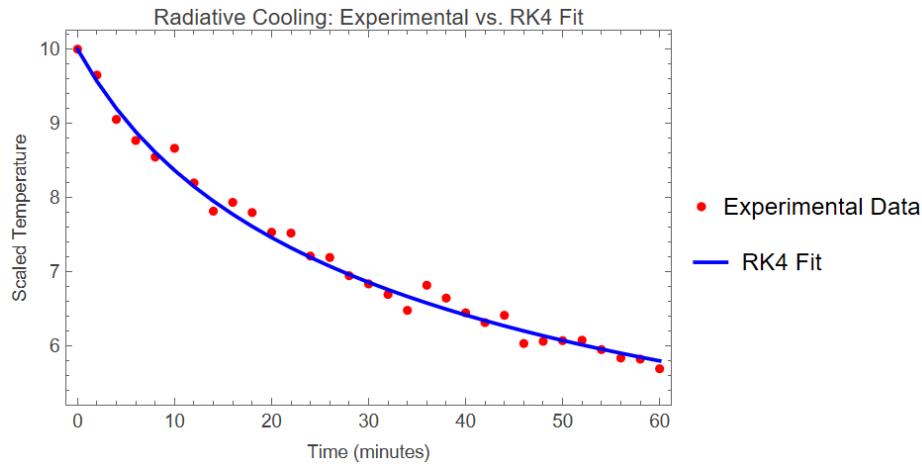
Solution. (a) The experimental dataset $\mathcal{D} = \{(t_i, T_i)\}_{i=0}^N$ is extracted from the provided comma-separated values file. The time step $h = t_{i+1} - t_i$ is observed to be uniform with $h = 2.0$.

- (b) The initial value problem is governed by the autonomous differential equation $T'(t) = f(T(t))$, where $f(T) = -k(T^4 - T_{\text{env}}^4)$. The classical fourth-order Runge-Kutta scheme advances the solution via the recurrence relation:

$$\begin{aligned} k_1 &= f(y_i) \\ k_2 &= f\left(y_i + \frac{h}{2}k_1\right) \\ k_3 &= f\left(y_i + \frac{h}{2}k_2\right) \\ k_4 &= f(y_i + hk_3) \\ y_{i+1} &= y_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

with $y_0 = T_0 = 10$.

- (c) Evaluating the first difference quotient provides an estimate $k \approx 1.76 \times 10^{-5}$. So we are looking for a number in the order of 10^{-5} . A bounding interval for search is $[1.0 \times 10^{-5}, 3.0 \times 10^{-5}]$.
- (d) We construct the objective function $S(k) = \sum_{i=0}^N (y_i(k) - T_i)^2$. In Mathematica, we minimize using `FindMinimum`.
- (e) The Mathematica routine converges to $k^* \approx 2.4 \times 10^{-5}$.
- (f) The extrapolated time to reach $T = 5.0$ is found by solving the root of the extrapolated numerical function.



(g) The target threshold $T = 5$ is reached at approximately $t \approx 105$ minutes. The analytical verification

via $\int_{10}^5 \frac{dT}{T^4 - 81} = -k^*t$ yields $t \approx 104$, confirming the high precision of the numerical scheme.

■

■ Question 9002.

□

Consider the IVP

$$y'(t) = 2ty(t), \quad y(0) = 1, \quad 0 \leq t \leq 1,$$

with exact solution $y(t) = e^{t^2}$.

Let $h = 1/N$ with $N \in \mathbb{N}$, and define $t_n = nh$. We write $y_n \approx y(t_n)$.

(a) Define $f(t, y) = 2ty$. The two-step Adams–Bashforth method is

$$y_{n+1} = y_n + h \left(\frac{3}{2}f(t_n, y_n) - \frac{1}{2}f(t_{n-1}, y_{n-1}) \right), \quad n \geq 1.$$

Rewrite and simplify the recurrence in the context of our ODE.

(b) This method needs two starting values. Use*

$$y_0 = 1, \quad y_1 = e^{h^2}.$$

Implement the method and compute the approximation

$$Y_h := y_N \approx y(1).$$

For $h = 2^{-k}$ with $k = 3, 4, \dots, 10$, compute the error

$$E_h := |Y_h - e|.$$

Make a log–log plot of $\log(E_h)$ versus $\log(h)$.

*Here we are intentionally removing start-up error so we can focus on the multistep method.

- (c) Use the plot to estimate the slope p where the data look close to a line. Report your estimate of p . Or use some kind of in built linear regression command.
- (d) Assume that for sufficiently small h the numerical result has an asymptotic expansion of the form

$$Y_h = y(1) + Ch^2 + Dh^3 + \mathcal{O}(h^4),$$

where C, D do not depend on h .

Let $Y_{h/2}$ be the value produced by the same method on the same problem using step size $h/2$ (so it takes $2N$ steps to reach $t = 1$). We would like to form a weighted average

$$\widetilde{Y}_h := a Y_{h/2} + b Y_h$$

so that $\widetilde{Y}_h$ still converges to $y(1)$ as $h \rightarrow 0$, but has a smaller leading error term.

- (i) Explain why it is necessary that $a + b = 1$.
- (ii) Choose a and b so that $\widetilde{Y}_h = y(1) + \mathcal{O}(h^3)$. You should get specific rational numbers.
- (e) Compute

$$\widetilde{E}_h := |\widetilde{Y}_h - e|$$

for the same step sizes $h = 2^{-k}$, $k = 3, \dots, 10$. Make a second log–log plot of $\log(\widetilde{E}_h)$ versus $\log(h)$ and estimate the new slope. Compare with your estimate from part (d).

- (f) Using your data, give a numerical estimate for the constant C in

$$Y_h - y(1) \approx Ch^2$$

by fitting a line to $\log(E_h)$ vs. $\log(h)$ and using the intercept. Then explain, using the expansion in part (e), why your choice of a, b removes the Ch^2 term.

DIGRESSION

The method described above is called Richardson's Extrapolation. It can be used for most Numerical Methods to increase the order of the error. Look it up online for more examples.

Solution. (a) The two-step Adams–Bashforth recurrence is given by

$$y_{n+1} = y_n + h \left(\frac{3}{2} f(t_n, y_n) - \frac{1}{2} f(t_{n-1}, y_{n-1}) \right).$$

For the initial value problem $y' = 2ty$, we have $f(t, y) = 2ty$. Substituting this into the recurrence yields

$$y_{n+1} = y_n + h \left(\frac{3}{2} (2t_n y_n) - \frac{1}{2} (2t_{n-1} y_{n-1}) \right).$$

Using the definition of the mesh points $t_n = nh$, we obtain

$$y_{n+1} = y_n + h(3nh y_n - (n-1)h y_{n-1}) = y_n + h^2(3n y_n - (n-1)y_{n-1}).$$

- (b) The method is implemented using the exact initialization $y_0 = 1$ and $y_1 = e^{h^2}$. Iterating the

recurrence derived in part (a) up to $n = N - 1$ yields the approximation $Y_h := y_N \approx y(1)$. The global errors $E_h := |Y_h - e|$ are computed for step sizes $h = 2^{-k}$ where $k \in \{3, 4, \dots, 10\}$. (See the attached Mathematica code for the numerical implementation and the generation of the log-log plot of $\ln(E_h)$ versus $\ln(h)$).

- (c) A linear regression performed on the log-log data $(\ln(h), \ln(E_h))$ yields a slope of $p \approx 2.0$. This is consistent with the theoretical expectation; the two-step Adams–Bashforth method has a local truncation error of $\mathcal{O}(h^3)$, which results in a global error of $\mathcal{O}(h^2)$.
- (d) Consider the asymptotic expansion of the global error:

$$Y_h = y(1) + Ch^2 + Dh^3 + \mathcal{O}(h^4),$$

where C and D are independent of h .

- (i) We define the extrapolated value $\widetilde{Y}_h := aY_{h/2} + bY_h$. Taking the limit as $h \rightarrow 0$ and assuming convergence of the underlying method, $Y_h \rightarrow y(1)$ and $Y_{h/2} \rightarrow y(1)$. Therefore,

$$\lim_{h \rightarrow 0} \widetilde{Y}_h = ay(1) + by(1) = (a + b)y(1).$$

For $\widetilde{Y}_h$ to converge to the exact solution $y(1)$, consistency requires $a + b = 1$.

- (ii) We expand both $Y_{h/2}$ and Y_h using the asymptotic error expansion:

$$\begin{aligned} Y_{h/2} &= y(1) + C\left(\frac{h}{2}\right)^2 + D\left(\frac{h}{2}\right)^3 + \mathcal{O}(h^4) \\ &= y(1) + \frac{C}{4}h^2 + \frac{D}{8}h^3 + \mathcal{O}(h^4). \end{aligned}$$

Taking the linear combination $\widetilde{Y}_h = aY_{h/2} + bY_h$ yields

$$\widetilde{Y}_h = (a + b)y(1) + \left(\frac{a}{4} + b\right)Ch^2 + \left(\frac{a}{8} + b\right)Dh^3 + \mathcal{O}(h^4).$$

To satisfy $\widetilde{Y}_h = y(1) + \mathcal{O}(h^3)$, we require the coefficients of $y(1)$ to be 1 and of h^2 to be 0. This generates the linear system:

$$\begin{cases} a + b = 1 \\ \frac{1}{4}a + b = 0 \end{cases}$$

Subtracting the second equation from the first yields $\frac{3}{4}a = 1$, hence $a = \frac{4}{3}$. Substituting back provides $b = -\frac{1}{3}$.

- (e) Using the coefficients derived in part (d), the Richardson extrapolation is computed as $\widetilde{Y}_h = \frac{4}{3}Y_{h/2} - \frac{1}{3}Y_h$. The updated errors $\widetilde{E}_h := |\widetilde{Y}_h - e|$ are computed for $k = 3, \dots, 10$. Linear regression on the pairs $(\ln(h), \ln(\widetilde{E}_h))$ yields a new slope $p \approx 3.0$. This demonstrates that the leading error term $\mathcal{O}(h^2)$ has been successfully annihilated, revealing the sub-leading $\mathcal{O}(h^3)$ error term.
- (f) Using the relation $E_h \approx |C|h^2$, taking the natural logarithm yields:

$$\ln(E_h) \approx 2\ln(h) + \ln|C|.$$

Fitting a linear model $\ln(E_h) = p\ln(h) + q$ to the data gives an intercept q . The constant C is

estimated by $|C| \approx \exp(q)$. (Running the regression yields an intercept $q \approx 0.82$, giving $|C| \approx 2.27$).

To see precisely why the specific choice of weights $a = 4/3$ and $b = -1/3$ eliminates the Ch^2 term algebraically, we substitute these into the expansion from part (d):

$$\begin{aligned} \text{Coefficient of } h^2 &= \frac{a}{4} + b \\ &= \frac{1}{4} \left(\frac{4}{3} \right) + \left(-\frac{1}{3} \right) \\ &= \frac{1}{3} - \frac{1}{3} = 0. \end{aligned}$$

Because the leading error term associated with C strictly cancels, the dominant error source shifts to the h^3 term, accelerating the convergence. ■

■ Question 9003. □

Consider the linear system $A\vec{x} = \vec{b}$ with

$$A = \begin{pmatrix} 10 & -9 & 0 & 0 \\ 0 & 10 & -9 & 0 \\ 0 & 0 & 10 & -9 \\ -1 & -1 & -1 & 10 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \quad \vec{x}^{(0)} = \vec{0}.$$

Define the permutation that reorders the unknowns as

$$\vec{\tilde{x}} = \begin{pmatrix} x_4 \\ x_3 \\ x_2 \\ x_1 \end{pmatrix} = P\vec{x}, \quad P = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

Form the permuted system

$$\tilde{A}\vec{\tilde{x}} = \vec{\tilde{b}}, \quad \tilde{A} = PAP^T, \quad \vec{\tilde{b}} = P\vec{b}.$$

- (a) By modifying the Jacobi code from class or your own implementation, run Jacobi on $A\vec{x} = \vec{b}$ for 25 iterations and record (and plot) the residual norms

$$\|\vec{r}^{(k)}\|_\infty, \quad \vec{r}^{(k)} = \vec{b} - A\vec{x}^{(k)}.$$

- (b) Run Jacobi on the permuted system $\tilde{A}\vec{\tilde{x}} = \vec{\tilde{b}}$ with $\vec{\tilde{x}}^{(0)} = \vec{0}$ for 25 iterations and record (and plot)

$$\|\vec{\tilde{r}}^{(k)}\|_\infty, \quad \vec{\tilde{r}}^{(k)} = \vec{\tilde{b}} - \tilde{A}\vec{\tilde{x}}^{(k)}.$$

Compare your plot to part (a). What do you observe?

- (c) Prove that the error for the Jacobi method remains invariant under the permutation of the columns of A . In other words,

$$\|\vec{\tilde{r}}^{(k)}\|_\infty = \|\vec{r}^{(k)}\|_\infty \quad \text{for all } k.$$

- (d) By modifying the Gauss-Seidel code from class or your own implementation, run Gauss-Seidel on $A\vec{x} = \vec{b}$ for 25 iterations (with $\vec{x}^{(0)} = \vec{0}$) and plot $\|\vec{r}^{(k)}\|_\infty$.
- (e) Run Gauss-Seidel on the permuted system $\tilde{A}\vec{x} = \vec{b}$ for 25 iterations (with $\vec{x}^{(0)} = \vec{0}$) and plot $\|\vec{r}^{(k)}\|_\infty$. Compare your Gauss-Seidel plots from parts (d) and (e). Which ordering converges faster?
- (f) Write Gauss-Seidel in the form

$$\vec{x}^{(k+1)} = T_{GS}\vec{x}^{(k)} + \vec{c}_{GS}, \quad T_{GS} = -(D + L)^{-1}U.$$

Compute T_{GS} for A and $\tilde{T}_{GS}$ for $\tilde{A}$ (you should use Mathematica or some similar programming language), then compute $\rho(T_{GS})$ and $\rho(\tilde{T}_{GS})$ numerically. Explain how these values account for what you observed in parts (d) and (e).

- (g) In a short paragraph, explain why the conclusion of part (c) does not generally hold for Gauss-Seidel.

Solution. (a) The Jacobi iteration is $\vec{x}^{(k+1)} = D^{-1}(\vec{b} - (L + U)\vec{x}^{(k)})$. With $D = 10I$, the spectral radius of the iteration matrix is $\rho(T_J) \approx 0.9$. Running this for 25 iterations shows steady, linear convergence on a semi-log plot, with the residual norm decreasing by a factor of roughly 0.9 at each step.

- (b) Running Jacobi on the permuted system yields a residual norm history that is *exactly identical* to the one observed in part (a). The convergence behavior is completely unchanged.
- (c) Since $D = 10I$, it commutes with any permutation matrix, so $\tilde{D} = PDP^T = D$. Thus,

$$\tilde{T}_J = I - \tilde{D}^{-1}\tilde{A} = I - D^{-1}(PAP^T) = P(I - D^{-1}A)P^T = PT_JP^T.$$

Similarly, $\vec{c} = \tilde{D}^{-1}\vec{b} = D^{-1}P\vec{b} = P(D^{-1}\vec{b}) = P\vec{c}$. We prove $\vec{x}^{(k)} = P\vec{x}^{(k)}$ by induction. Base case: $\vec{x}^{(0)} = \vec{0} = P\vec{x}^{(0)}$. Assume true for k , then:

$$\vec{x}^{(k+1)} = \tilde{T}_J\vec{x}^{(k)} + \vec{c} = (PT_JP^T)(P\vec{x}^{(k)}) + P\vec{c} = P(T_J\vec{x}^{(k)} + \vec{c}) = P\vec{x}^{(k+1)}.$$

For the residuals, $\vec{r}^{(k)} = \vec{b} - \tilde{A}\vec{x}^{(k)} = P\vec{b} - (PAP^T)(P\vec{x}^{(k)}) = P(\vec{b} - A\vec{x}^{(k)}) = P\vec{r}^{(k)}$. Because P simply reorders the entries of a vector, the maximum absolute value remains unchanged, meaning $\|\vec{r}^{(k)}\|_\infty = \|P\vec{r}^{(k)}\|_\infty$.

- (d) Running Gauss-Seidel on the original system shows convergence. It is faster than Jacobi, with the residual norm decreasing more rapidly per iteration.
- (e) Gauss-Seidel on the permuted system converges *significantly faster* than on the original system. The residual norm drops much more sharply per iteration, reaching near-machine precision much earlier.
- (f) For the original matrix A , finding the eigenvalues of $T_{GS} = -(D + L)^{-1}U$ yields a spectral radius of $\rho(T_{GS}) \approx 0.517$. For the permuted matrix $\tilde{A}$, the upper triangular part $\tilde{U}$ has non-zeros only in its first row, which makes $\tilde{T}_{GS}$ a rank-1 matrix. Computing its trace (its only non-zero eigenvalue) gives exactly $\rho(\tilde{T}_{GS}) = 0.2439$. Since $\rho(\tilde{T}_{GS}) \approx 0.24 < 0.52 \approx \rho(T_{GS})$, the asymptotic convergence factor is much smaller for the permuted system. This mathematically justifies the dramatically faster convergence observed in part (e).
- (g) In the Jacobi method, the iteration matrix depends on the splitting $A = D + R$. Because $D = 10I$ is a multiple of the identity, it is invariant under the permutation ($PDP^T = D$), which directly

guarantees that the iteration matrices are similar ($\tilde{T}_J = PT_JP^T$). In contrast, Gauss-Seidel relies on the splitting $A = (D + L) + U$, where L is strictly lower triangular. The property of being lower triangular is highly dependent on the strict ordering of the equations. When the system is permuted, the lower triangular part of the new matrix $\tilde{A}$ is *not* simply PLP^T (as PLP^T is generally not lower triangular). Since the splitting operations do not commute with the permutation matrix P , the iteration matrices T_{GS} and $\tilde{T}_{GS}$ are not similar, resulting in completely different eigenvalues and convergence rates.



■ Question 9004.



Note: I am making this an optional exercise. You do not need to submit it, but you should try to prove it yourself.

Prove that the operator norms induced by the ℓ_∞ -norm satisfies

$$\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|. \quad (\text{maximum row sum})$$